

Support Vector Machines

Support Vector Machine (SVM)

SVM (Vapnik, 1995) is a classification tool that was originally invented in the field of pattern recognition. It is now popular in the statistical community. We consider the 2-class problem.

Maximal margin classifier (linear boundary, separable case)

One can compute the (perpendicular) distance from each training observation to a given separating hyperplane; the smallest such distance is the minimal distance from the observations to the hyperplane, and is known as margin. The maximal margin hyperplane is the separating hyperplane for which the margin is the largest - that is, it is the hyperplane that has the farthest minimum distance to the training observations. In a sense, it represents the mid-line of the widest “slab” that we can inset between the two classes. One can then classify a test observation based on which side of the maximal margin hyperplane it lies - maximal margin classifier. In the plot, we see that three training observations are equidistant from the maximal margin hyperplane and lie along the dashed lines indicating the width of the margin. These three observations are known as support vectors. Interestingly, the maximal margin hyperplane depends directly on the support, but not on the other observations.

Construction: $x_1, \dots, x_n \in R^p$, $y_1, \dots, y_n \in \{-1, 1\}$

Find $f(x) = \beta_0 + \beta_1 x_1 + \dots + \beta_p x_p$ as the solution to the following optimization problem:

Find $(\beta_0, \beta_1, \dots, \beta_p)$ that maximizes M subject to $\sum_{j=1}^p \beta_j^2 = 1$, $y_i(\beta_0 + \beta_1 x_{i1} + \dots + \beta_p x_{ip}) \geq M$ for $i = 1, \dots, n$.

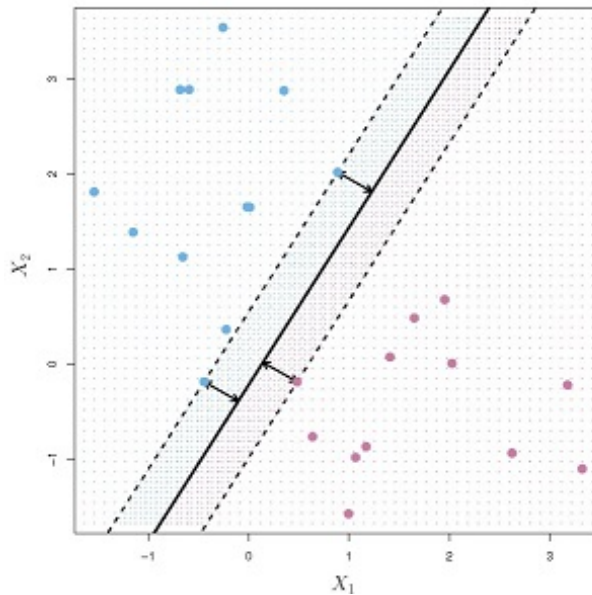


Figure 1: Maximal margin classifier

The maximal margin hyperplane is extremely sensitive to a change in a single observation - it may have overfit the training data. We might be willing to consider a classifier based on a hyperplane that does not

exactly separate the two classes, in the interest of greater robustness to individual observations and better classification of most of the training observations.

Exercise #1:

Make 2D data that is not separable by a linear boundary. Make 2D simulated data that IS separable by a linear boundary. Plot both data sets with different colors for the classes.

Exercise #2

Add a line into your plot from Exercise 1 showing the line separating the data into two classes. You should be able to figure out what this line based on how you generated the data. The function `abline` can be used to add the line to the plot.

Exercise #3

Note that the distance from each point to the hyperplane is given by $y_i(\beta_0 + \beta_1 x_{i1} + \dots + \beta_p x_{ip})$. Calculate this distance for every point from exercise #1 and find the point(s) with the smallest distance. These points are the support vectors.

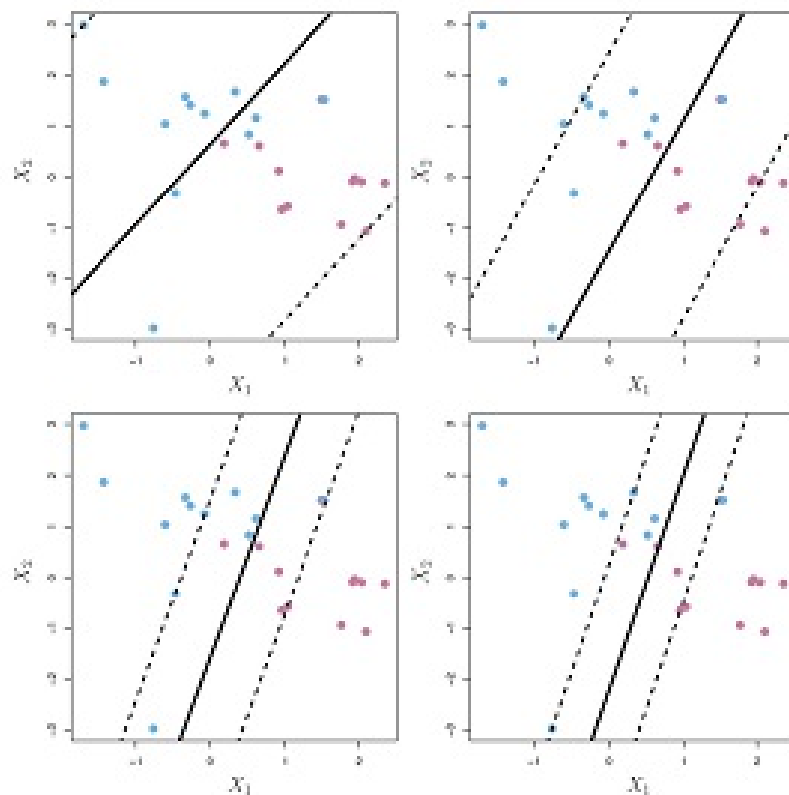
Support vector classifier (linear boundary, non-separable case)

Rather than seeking the largest possible margin, we instead allow some observations to be on the incorrect side of the margin, or even the incorrect side of the hyperplane - soft margin classifier. It attempts to achieve greater robustness to individual observations and better classification of most of the training observations.

Construction: Find $(\beta_0, \beta_1, \dots, \beta_p)$ that maximizes M subject to

$$\sum_{j=1}^p \beta_j^2 = 1, \quad y_i(\beta_0 + \beta_1 x_{i1} + \dots + \beta_p x_{ip}) \geq M(1 - \epsilon_i), \quad \epsilon_i \geq 0, \quad \sum_{i=1}^n \epsilon_i \leq C,$$

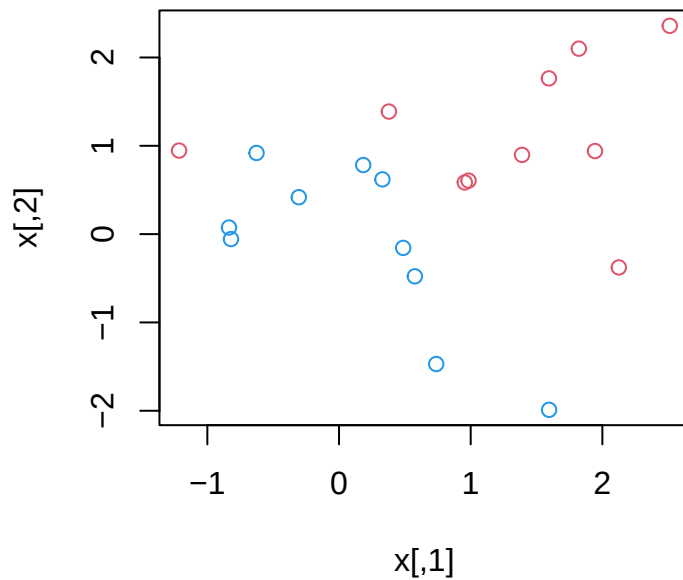
for $i = 1, \dots, n$ where C is nonnegative tuning parameter, and $\epsilon_1, \dots, \epsilon_n$ are slack variables that allow individual observations to be on the wrong side of the margin or the hyperplane. The slack variable ϵ_i tells us where the i th observation is located, relative to the hyperplane and relative to the margin. If $\epsilon_i = 0$ then the i th observation is on the correct side of the margin. If $\epsilon_i > 0$ then the i th observation is on the wrong side of the margin, and we say that the i th observation has violated the margin. If $\epsilon_i > 1$ then it is on the wrong side of the hyperplane. C bounds the sum of ϵ_i 's, and so it determines the number and severity of the violations to the margin (and to the hyperplane) that we will tolerate. When C is small, we seek narrow margins that are rarely violated; this amounts to a classifier that is highly fit to the data, which may have low bias but high variance. On the other hand, when C is larger, the margin is wider and we allow more violations to it; this amounts to fitting the data less hard and obtaining a classifier that is potentially more biased but may have lower variance. Since the support vector classifier's decision rule is based only on a potentially small subset of the training observations (support vectors), it is quite robust to the behavior of observations that are far away from the hyperplane.



Support vector classifier

Start by making some data:

```
set.seed(1)
x=matrix(rnorm(20*2), ncol=2)
y=c(rep(-1,10), rep(1,10))
x[y==1,]=x[y==1,] + 1 #shift the X values if y=1
plot(x, col=(3-y)) # not separable
```



This data is not separable.

We use the `svm` function in R for fitting a support vector machine:

```
library(e1071)
?svm
```

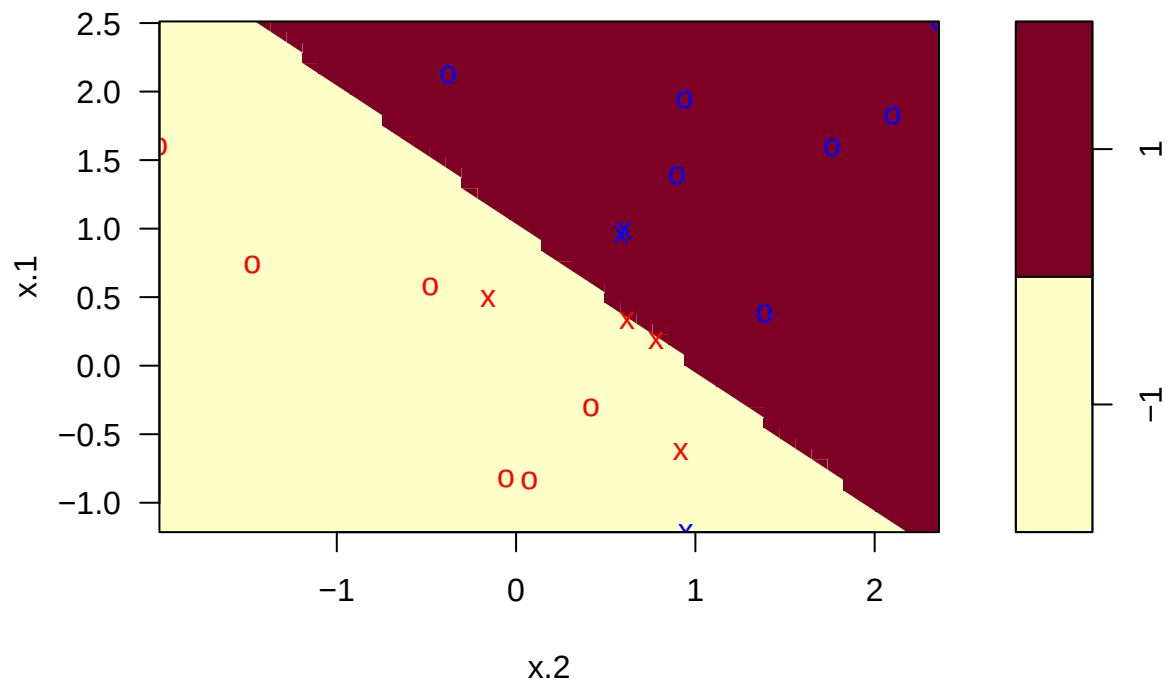
```
## starting httpd help server ... done
```

The `svm` function has similar syntax to many model fitting functions in R.

Apply to the data. Specify colors for classes as blue and red.

```
dat=data.frame(x=x, y=as.factor(y)) # response should be a factor
library(e1071)
# linear boundary, C=10, no standardization
svmfit=svm(y~., data=dat, kernel="linear", cost=10,scale=FALSE)
plot(svmfit, dat,symbolPalette=c("red","blue")) #one misclassification
```

SVM classification plot



```
summary(svmfit)
```

```
##
## Call:
## svm(formula = y ~ ., data = dat, kernel = "linear", cost = 10, scale = FALSE)
##
##
## Parameters:
##   SVM-Type:  C-classification
##   SVM-Kernel: linear
##       cost:  10
##
## Number of Support Vectors:  7
##
##   ( 4 3 )
##
## Number of Classes:  2
##
## Levels:
##   -1 1
```

The plot shows the classification regions. Note that x1 and x2 are flipped. The symbol 0 represents data points that are not support vectors. The symbol x represents points that are support vector.

The element **SV** contains these support vectors. These are the data points that are within the margin or on the wrong side of the boundary. **SV** gives the coordinates of these points:

```
svmfit$SV
```

```
##           x.1           x.2
## 1 -0.6264538  0.9189774
## 2  0.1836433  0.7821363
## 5  0.3295078  0.6198257
## 7  0.4874291 -0.1557955
## 14 -1.2146999  0.9461950
## 16  0.9550664  0.5850054
## 17  0.9838097  0.6057100
```

The element `index` gives the indexes (row number) of the support vector points. Below, I use this to add the y-values to the support vectors:

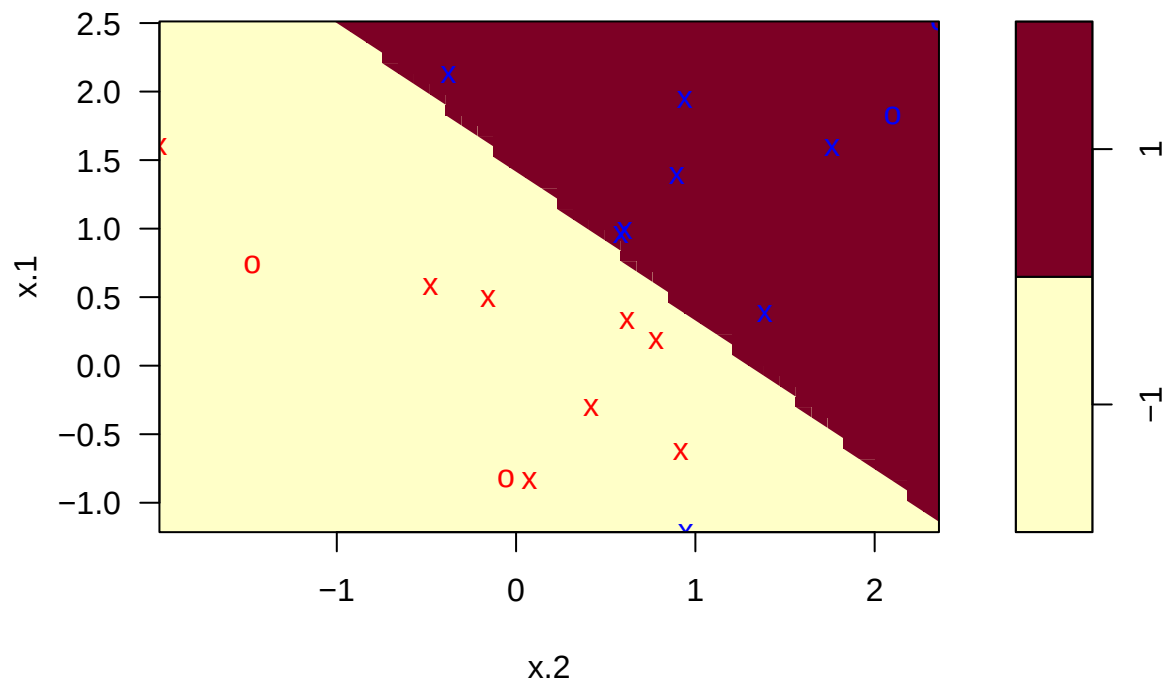
```
cbind(svmfit$SV,y[svmfit$index])
```

```
##           x.1           x.2
## 1 -0.6264538  0.9189774 -1
## 2  0.1836433  0.7821363 -1
## 5  0.3295078  0.6198257 -1
## 7  0.4874291 -0.1557955 -1
## 14 -1.2146999  0.9461950  1
## 16  0.9550664  0.5850054  1
## 17  0.9838097  0.6057100  1
```

Apply with a smaller cost:

```
svmfit=svm(y~., data=dat, kernel="linear", cost=0.1,scale=FALSE) #C=0.1
plot(svmfit, dat,symbolPalette=c("red","blue"))
```

SVM classification plot



```
svmfit$index
```

```
## [1] 1 2 3 4 5 7 9 10 12 13 14 15 16 17 18 20
```

There are more support vectors when the cost is lower.

Exercise #4: Apply the `svm` function to your non-separable data from exercise 1. Identify the support vectors.

Tuning with cross validation

Here I used 10-fold cross validation using the `tune` function.

```
?tune
```

```
set.seed(1)
tune.out=tune(svm,y~.,data=dat,kernel="linear",ranges=list(cost=c(0.001, 0.01, 0.1, 1,5,10,100)))
summary(tune.out) #C=0.1 is the best
```

```
##
## Parameter tuning of 'svm':
##
## - sampling method: 10-fold cross validation
##
## - best parameters:
##   cost
##   0.1
##
```

```
## - best performance: 0.05
##
## - Detailed performance results:
##   cost error dispersion
## 1 1e-03 0.55 0.4377975
## 2 1e-02 0.55 0.4377975
## 3 1e-01 0.05 0.1581139
## 4 1e+00 0.15 0.2415229
## 5 5e+00 0.15 0.2415229
## 6 1e+01 0.15 0.2415229
## 7 1e+02 0.15 0.2415229

bestmod=tune.out$best.model
summary(bestmod)

##
## Call:
## best.tune(METHOD = svm, train.x = y ~ ., data = dat, ranges = list(cost = c(0.001,
##   0.01, 0.1, 1, 5, 10, 100)), kernel = "linear")
##
##
## Parameters:
##   SVM-Type:  C-classification
##   SVM-Kernel: linear
##   cost:      0.1
##
## Number of Support Vectors: 16
##
## ( 8 8 )
##
##
## Number of Classes: 2
##
## Levels:
## -1 1
```

Prediction

Next, we do prediction using the tuned parameter:

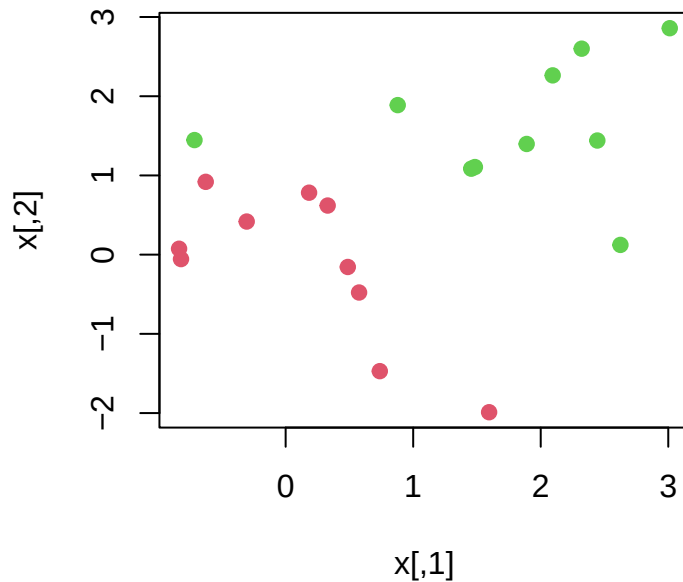
```
xtest=matrix(rnorm(20*2), ncol=2)
ytest=sample(c(-1,1), 20, rep=TRUE)
xtest[ytest==1,]=xtest[ytest==1,] + 1
testdat=data.frame(x=xtest, y=as.factor(ytest))
ypred=predict(bestmod,testdat)
table(predict=ypred, truth=testdat$y)
```

```
##      truth
## predict -1 1
##      -1  9 1
##      1  2 8
```

The model performs well in this case.

Linearly separable case

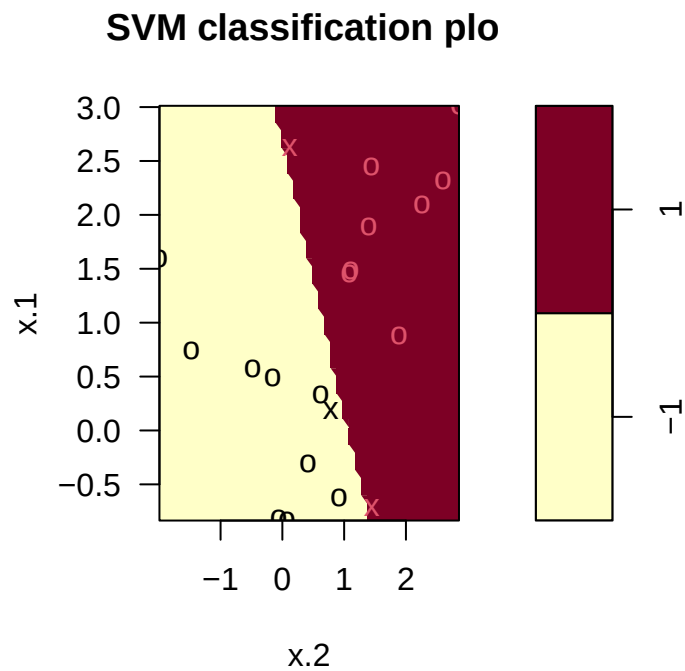
```
x[y==1,]=x[y==1,]+0.5
plot(x, col=(y+5)/2, pch=19)
```



```
dat=data.frame(x=x,y=as.factor(y))
svmfit=svm(y~., data=dat, kernel="linear", cost=1e5) #large C, poor generalization
summary(svmfit)
```

```
##
## Call:
## svm(formula = y ~ ., data = dat, kernel = "linear", cost = 1e+05)
##
##
## Parameters:
##   SVM-Type:  C-classification
##   SVM-Kernel: linear
##       cost:  1e+05
##
## Number of Support Vectors:  3
##
##   ( 1 2 )
##
##
## Number of Classes:  2
##
## Levels:
##   -1 1
```

```
plot(svmfit, dat)
```

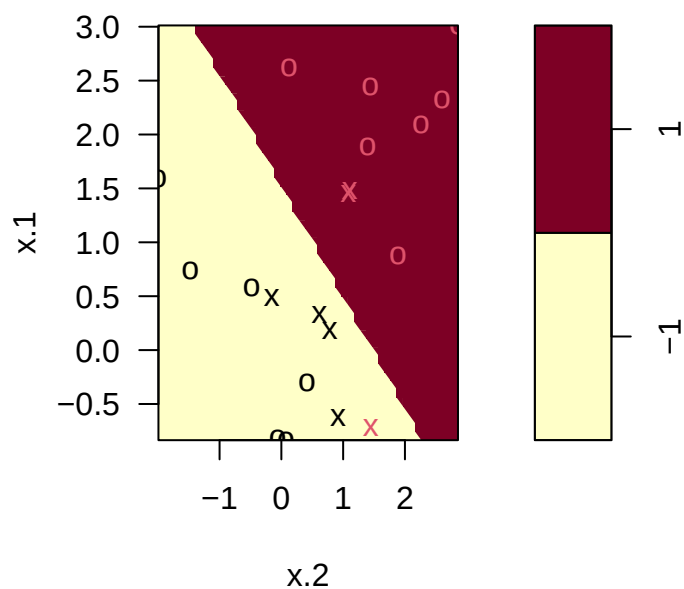


```
svmfit=svm(y~., data=dat, kernel="linear", cost=1)
summary(svmfit)
```

```
##
## Call:
## svm(formula = y ~ ., data = dat, kernel = "linear", cost = 1)
##
##
## Parameters:
##   SVM-Type:  C-classification
##   SVM-Kernel: linear
##       cost:  1
##
## Number of Support Vectors:  7
##
## ( 4 3 )
##
##
## Number of Classes:  2
##
## Levels:
##   -1  1
```

```
plot(svmfit,dat)
```

SVM classification plo



Support Vector Machines (non-linear decision boundaries)

It allows general decision boundaries by introducing a nonlinear map.

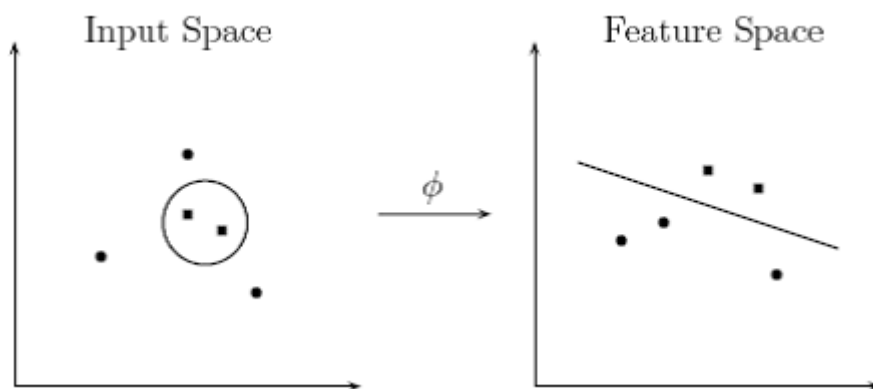


Figure 2: Non-linear SVM

The linear support vector classifier can be represented as

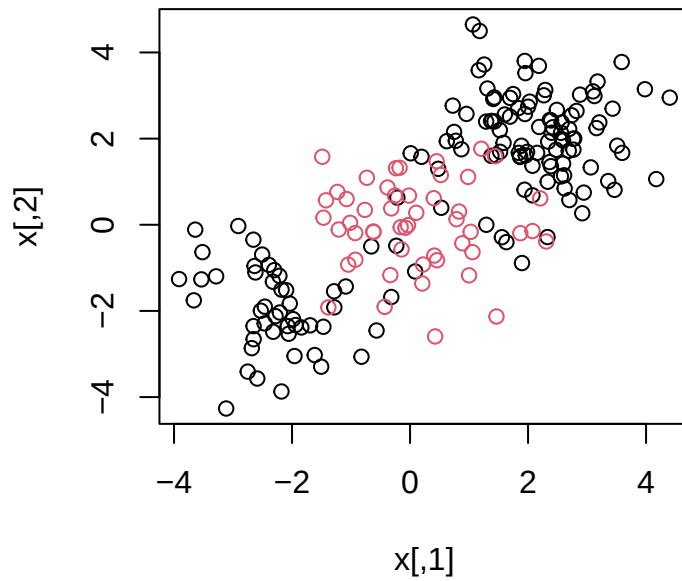
$$f(x) = \beta_0 + \sum_{i=1}^n \alpha_i \langle x, x_i \rangle,$$

where there are n parameters α_i , one per training observation. SVM replaces the inner product with a generalization of the inner product of the form:

$$f(x) = \beta_0 + \sum_{i=1}^n \alpha_i K(x, x_i)$$

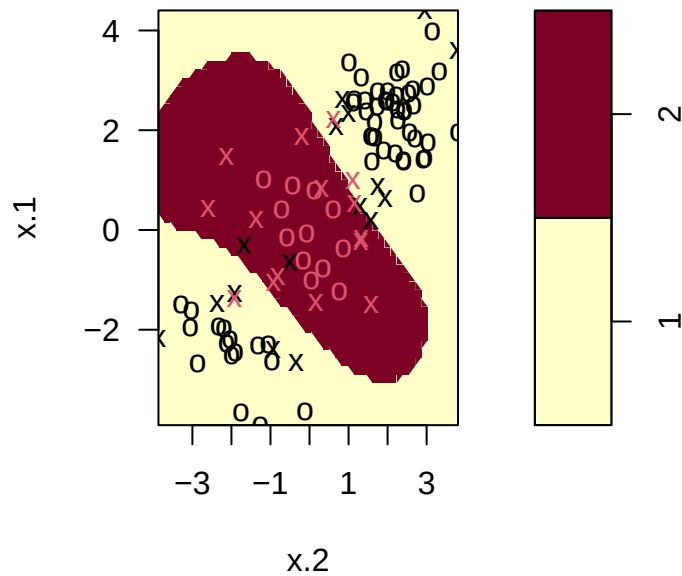
where K is a kernel. Popular choices are polynomial and radial kernel functions.

```
set.seed(1)
x=matrix(rnorm(200*2), ncol=2)
x[1:100,]=x[1:100,]+2
x[101:150,]=x[101:150,]-2
y=c(rep(1,150),rep(2,50))
dat=data.frame(x=x,y=as.factor(y))
plot(x, col=y)
```



```
train=sample(200,100)
svmfit=svm(y~., data=dat[train,], kernel="radial", gamma=1, cost=1)
plot(svmfit, dat[train,]) #nonlinear boundary
```

SVM classification plo

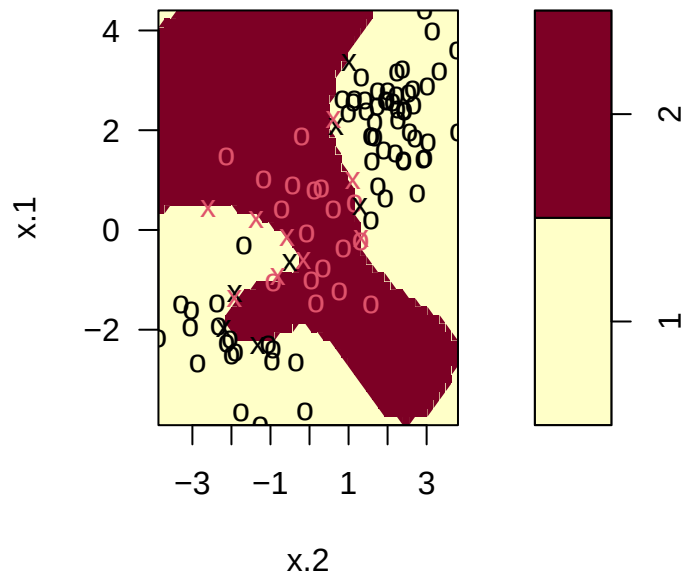


```
summary(svmfit)
```

```
##
## Call:
## svm(formula = y ~ ., data = dat[train, ], kernel = "radial", gamma = 1,
##      cost = 1)
##
##
## Parameters:
##   SVM-Type:  C-classification
##   SVM-Kernel: radial
##      cost:  1
##
## Number of Support Vectors:  31
##
##   ( 16 15 )
##
##
## Number of Classes:  2
##
## Levels:
##   1 2
```

```
svmfit=svm(y~., data=dat[train,], kernel="radial",gamma=1,cost=1e5)
plot(svmfit,dat[train,]) #less error, more irregular boundaries
```

SVM classification plo



```
set.seed(1)
tune.out=tune(svm, y~., data=dat[train,], kernel="radial", ranges=list(cost=c(0.1,1,10,100,1000),gamma=
summary(tune.out) #C=1, gamma=2
```

```
##
## Parameter tuning of 'svm':
##
## - sampling method: 10-fold cross validation
##
## - best parameters:
##   cost gamma
##   1 0.5
##
## - best performance: 0.07
##
## - Detailed performance results:
##   cost gamma error dispersion
## 1 1e-01 0.5 0.26 0.15776213
## 2 1e+00 0.5 0.07 0.08232726
## 3 1e+01 0.5 0.07 0.08232726
## 4 1e+02 0.5 0.14 0.15055453
## 5 1e+03 0.5 0.11 0.07378648
## 6 1e-01 1.0 0.22 0.16193277
## 7 1e+00 1.0 0.07 0.08232726
## 8 1e+01 1.0 0.09 0.07378648
## 9 1e+02 1.0 0.12 0.12292726
## 10 1e+03 1.0 0.11 0.11005049
## 11 1e-01 2.0 0.27 0.15670212
## 12 1e+00 2.0 0.07 0.08232726
```

```
## 13 1e+01 2.0 0.11 0.07378648
## 14 1e+02 2.0 0.12 0.13165612
## 15 1e+03 2.0 0.16 0.13498971
## 16 1e-01 3.0 0.27 0.15670212
## 17 1e+00 3.0 0.07 0.08232726
## 18 1e+01 3.0 0.08 0.07888106
## 19 1e+02 3.0 0.13 0.14181365
## 20 1e+03 3.0 0.15 0.13540064
## 21 1e-01 4.0 0.27 0.15670212
## 22 1e+00 4.0 0.07 0.08232726
## 23 1e+01 4.0 0.09 0.07378648
## 24 1e+02 4.0 0.13 0.14181365
## 25 1e+03 4.0 0.15 0.13540064
```

```
table(true=dat[-train,"y"], pred=predict(tune.out$best.model,newx=dat[-train,]))
```

```
##      pred
## true 1  2
##      1 54 23
##      2 17  6
```

ROC (Receiver Operating Characteristics) curve

ROC curve is a popular graphic for simultaneously displaying the two types of errors for all possible thresholds. It traces out two types of error as one varies the threshold value for the posterior probability. The true positive rate is the sensitivity: the fraction of events that are correctly identified, using a given threshold value. The false positive rate is 1-specificity: the fraction of non-events that we classify incorrectly as events, using that same threshold value. The overall performance of a classifier, summarized over all possible thresholds, is given by the area under the (ROC) curve (AUC). An ideal ROC curve will hug the top left corner, indicating a high true positive rate and a low false positive rate, so the larger the AUC the better the classifier. The dotted line represents the “no information” classifier.

Test			
True	Non-event(H0)	Non-event(H0)	Event(H1)
Non-event(H0)	True Negative	True Negative	False Positive
Event(H1)	False Negative	False Negative	True Positive

Sensitivity=True Positive Rate= $P(\text{Reject } H_0 | H_0 \text{ is false}) = (\text{True Positive}) / (\text{True Positive} + \text{False Negative})$
 Specificity= $P(\text{Not reject } H_0 | H_0 \text{ is true}) = (\text{True Negative}) / (\text{True Negative} + \text{False Positive})$
 False Positive Rate=1-specificity= $(\text{False Positive}) / (\text{True Negative} + \text{False Positive})$

SVM with multiple classes

```
set.seed(1)
x=rbind(x, matrix(rnorm(50*2), ncol=2))
y=c(y, rep(0,50))
x[y==0,2]=x[y==0,2]+2
dat=data.frame(x=x, y=as.factor(y))
par(mfrow=c(1,1))
plot(x,col=(y+1))
```

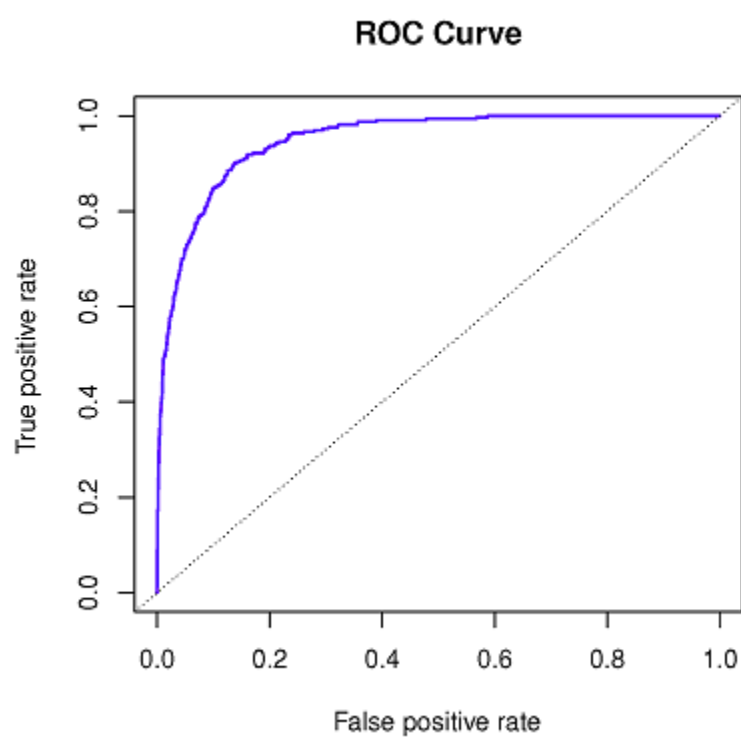
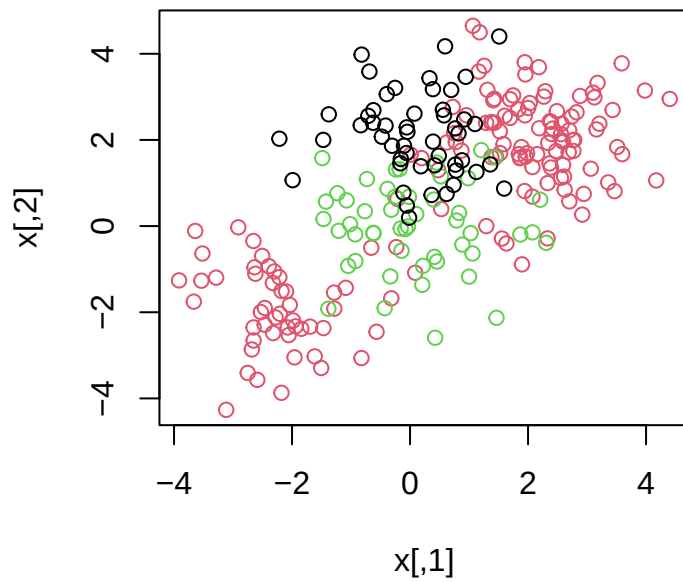


Figure 3: ROC Curve



```
svmfit=svm(y~., data=dat, kernel="radial", cost=10, gamma=1)
plot(svmfit, dat)
```

SVM classification plo

